

[--- CODE CELL ---]

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

[--- CODE CELL ---]

```
list_not_selected = []
a_set = {}
for i in range(1,1001):
    possible_values = np.arange(1,10000)
    a = np.random.choice(possible_values, size=10000, replace=True)
    a_set = set(a)
    not_selc = 10000 - len(a_set)
    per_for = (not_selc / 10000) * 100
    list_not_selected.append(per_for)
```

[--- CODE CELL ---]

```
max(list_not_selected)
```

[--- CODE CELL ---]

```
list_not_selected
```

[--- CODE CELL ---]

```
plt.hist(list_not_selected)
plt.show()
```

[--- CODE CELL ---]

```
df = pd.DataFrame(list_not_selected)
```

[--- CODE CELL ---]

```
df.describe()
```

[--- CODE CELL ---]

```
np_array = np.array(list_not_selected)
```

[--- CODE CELL ---]

```
np_array_means = np.mean(np_array)
np_array_means
```

[--- CODE CELL ---]

```
np_array_standar_dev = np.std(np_array)
np_array_standar_dev
```

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris

data = load_iris()
X = data.data
y = data.target
X = X[:100, :]
y = y[:100]
```

```

# no of folds
K = 5
folds = np.random.choice(K, size=X.shape[0], replace=True)
X_test = X[folds == 0]
y_test = y[folds == 0]
X_train = X[folds != 0]
y_train = y[folds != 0]

# Bagging model
from sklearn.ensemble import BaggingClassifier
# default base estimator is Decision Tree with default hyperparameters
clf = BaggingClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_proba = clf.predict_proba(X_test)

# get performance metrics
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
tp = cm[0,0]
tn = cm[1,1]
fp = cm[1,0]
fn = cm[0,1]

accuaracy = (tp + tn) / cm.sum()
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1_score = 2 * (precision * recall) / (precision + recall)

#####

# base model is Decision Tree with max_depth=3
from sklearn.tree import DecisionTreeClassifier
base_estimator = DecisionTreeClassifier(
    criterion='gini',
    max_depth=3,
    splitter='best')
clf = BaggingClassifier(estimator=base_estimator,
                        n_estimators=100)

clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
tp = cm[0,0]
tn = cm[1,1]
fp = cm[1,0]
fn = cm[0,1]

accuaracy = (tp + tn) / cm.sum()
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1_score = 2 * (precision * recall) / (precision + recall)
print(f"accurcary: {accuaracy}")

#####

# base model is KNN with k=3
from sklearn.neighbors import KNeighborsClassifier
base_estimator = KNeighborsClassifier(n_neighbors=3)
clf = BaggingClassifier(estimator=base_estimator,
                        n_estimators=100)

clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
tp = cm[0,0]
tn = cm[1,1]
fp = cm[1,0]
fn = cm[0,1]

```

```

accuracy = (tp + tn) / cm.sum()
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1_score = 2 * (precision * recall) / (precision + recall)
#####
mcc_formula = (tp*tn - fp*fn) / np.sqrt((tp + fp) * (tp + fn) * (tn + fp) * (tn + fn))
from sklearn.metrics import matthews_corrcoef
mcc = matthews_corrcoef(y_test, y_pred)

```

[--- CODE CELL ---]

```

rng = np.random.RandomState(42)
np.random.seed(42)
np.random.random(5)

```

[--- CODE CELL ---]

```

np.random.seed(42)
np.random.random(5)

```

[--- CODE CELL ---]